

CHECKING WHAT YOUR COMPILER SUPPORTS

Not every "C++23 compiler" supports all of C++23

THE PROBLEM

Compilers implement standards feature by feature

A new C++ standard is a huge list of features. Compiler teams don't ship every feature the day the standard is published - they land gradually, often over several compiler releases. "My compiler supports C++23" doesn't automatically mean *every* feature this course uses is available.

THE REFERENCE

cppreference's compiler support table

en.cppreference.com/w/cpp/compiler_support

tracks, feature by feature, which GCC/Clang/MSVC version added support for it.

If something from this course doesn't compile for you, this page is the first place to check *why*.

The standard flag

Even a compiler capable of C++23 needs to be told to target it:

GCC / Clang

```
g++ -std=c++23 main.cpp  
clang++ -std=c++23 main.cpp
```

MSVC

```
/std:c++latest
```

This course's `CMakeLists.txt` sets this for you via

`CMAKE_CXX_STANDARD 23` - you don't need to pass the flag by hand.

Don't just check a version number - try it

This lecture's `main.cpp` doesn't just print a compiler name - it uses a few C++20/23 features directly: `<print>`, concepts, and digit separators. If your setup can't compile it, the error message tells you exactly which feature is missing.

```
template <typename T>
concept Addable = requires(T a, T b) {
    a + b;
};
```

READING THE FILE

Build it and see for yourself

```
cmake -B build -G Ninja .  
cmake --build build  
./build/rooster
```

If it builds and prints the closing message, your toolchain is ready for the rest of this course. If it doesn't, the compiler error - not just a version string - tells you which feature to look up on cppreference.

IF IT DOESN'T COMPILE

Two ways forward

- **Upgrade your compiler/kit** - if your OS package manager or Qt Creator's Kit selection offers a newer GCC/Clang/MSVC.
- **Use Docker** - if upgrading isn't convenient, or you don't want to touch your system's installed toolchain at all.